

WIPRO LIMITED

EMBEDDED SYSTEMS & LINUX KERNEL ENGINEERING TRACK

INDIVIDUAL PROJECT DOCUMENTATION (EXECUTIVE SUMMARY)

VDevPulse (vdevpulse)

A High-Performance Linux Virtual Character Device Daemon for Real-Time Kernel Telemetry, Threshold Alerting, Interactive Query Protocols, and Top Process Diagnostics

Project Title:	VDevPulse Linux System Telemetry Daemon
Name:	Priyansu Das
Registration No:	2341004087
Target Platform:	Linux Embedded Systems / POSIX.1-2008
Core Standard:	ISO C++17 (Zero External Dependencies)
Repository:	github.com/priyansudas07/.../vdevpulse
Submission Date:	October 2026

1. Project Introduction, Problem Statement & Engineering Rationale

1.1 Project Description & Strategic Objectives

VDevPulse is a zero-external-dependency C++17 system daemon engineered for resource-constrained embedded Linux environments. It creates a virtual character device node (/tmp/vdevpulse) via POSIX named pipes, enabling both human engineers and automated supervisory watchdogs to monitor kernel vitals synchronously without runtime overhead.

- **Zero Runtime Dependencies:** Pure ISO C++17 standard library and native Linux POSIX system calls.
- **Sub-Millisecond Kernel Telemetry:** Samples CPU, RAM, load averages, and active threads directly from /proc in under 3 milliseconds.
- **Top Background Process Scanner:** Inspects active PIDs in /proc to identify and rank top physical RAM (VmRSS) consumers.
- **Synchronous Machine Interface:** Non-blocking FIFO I/O and text query protocols (GET_CPU , GET_TOP) for automated watchdogs.

1.2 Why I Built This Project (Personal Engineering Motivation)

During embedded systems development, I observed three major real-world constraints on headless edge hardware:

1. **Absence of GUI / Task Manager:** Embedded nodes (automotive ECUs, industrial IoT gateways) run headless without desktop display managers or GUI task managers. Diagnostics must occur via serial UART or IPC.
2. **Heavyweight Tooling Penalty:** Standard monitoring agents (Prometheus, Datadog) consume hundreds of megabytes of RAM. In 128MB–512MB RAM systems, these tools trigger Linux Out-Of-Memory (OOM) killer panics. Furthermore, spawning top or ps duplicates page tables via fork() and exec() , causing CPU spikes.
3. **Need for Automated Watchdog Nodes:** Supervisory daemons require a predictable, non-blocking device node to read health stats synchronously and trigger automated recovery without human intervention.

2. System Architecture & Subsystem Design



2.1 Core Subsystems Breakdown

- **DeviceManager:** Manages /tmp/vdevpulse FIFO with O_RDWR | O_NONBLOCK , preventing I/O stalls and broken pipe errors.
- **TelemetryMonitor:** Computes CPU tick deltas (/proc/stat), parses physical RAM (/proc/meminfo), tracks load averages (/proc/loadavg), and scans top VmRSS process memory consumers (/proc/[PID]/status).
- **Threshold Alert Engine:** Evaluates live telemetry against rules in vdev_policy.json to assign dynamic health states (HEALTHY , WARNING_CPU_OVERLOAD , WARNING_MEMORY_PRESSURE).
- **Logger Singleton:** Thread-safe logging with std::lock_guard<std::mutex> for ANSI console and disk auditing.

3. CLI Command Suite & Operational Modes

Command	Syntax	Operational Behavior
status	vdevpulse status [--json]	Instantaneous system telemetry dashboard (ANSI table or structured JSON).
top	vdevpulse top [limit]	Ranks active background processes by physical memory consumption (default: 5).
history	vdevpulse history	Displays tabular time-series snapshots from the 60-sample in-memory ring buffer.
query	vdevpulse query <CMD>	Synchronous query (GET_CPU , GET_MEM , GET_LOAD , GET_TOP , GET_HEALTH , PING).
ioctl	vdevpulse ioctl <CMD>	Dispatches virtual driver IOCTL codes (start , stop , reset , stats).
run	vdevpulse run [policy] [--json]	Continuous daemon loop streaming telemetry frames to /tmp/vdevpulse .
write	vdevpulse write <msg>	Transmits diagnostic message payload to the virtual character device node.

4. Automated Verification Matrix & Test Strategy

Unit testing is automated via CMake CTest on Linux, achieving a **100% pass rate across all 20 assertions**:

Test Suite	Test ID	Scope & Target Assertion	Verification Target	Result
Device Test (device_test.cpp)	TC-DEV-01	FIFO Device Node Creation & Open	mkfifo() & non-blocking fd	PASS
	TC-DEV-02	Diagnostic Payload Transmission	writeData() non-blocking	PASS
	TC-DEV-03	IOCTL State & Statistics Control	START, STOP, RESET, STATS	PASS
	TC-DEV-04	Interactive Query Protocol	PING, GET_CPU, GET_TOP	PASS
Telemetry Test (telemetry_test.cpp)	TC-TEL-01	ProcFS Memory & Load Parsing	MemAvailable, load_1m >= 0	PASS
	TC-TEL-02	Top Process VmRSS Scanner	getTopProcesses() sorted	PASS
	TC-TEL-03	Threshold Health Engine	Dynamic policy evaluation	PASS
	TC-TEL-04	JSON Serialization & Ring Buffer	Valid JSON + 60-sample deque	PASS

5. Conclusion & Future Roadmap

Conclusion: VDevPulse demonstrates that high-performance, real-time Linux telemetry monitoring, process inspection, and virtual character device driver emulation can be achieved cleanly in ISO C++17 with zero external runtime dependencies.

Future Extensions: (1) Porting to an in-tree Linux Kernel Module (/dev/vdevpulse), (2) Integrating eBPF for zero-copy CPU tracing, (3) Direct CAN bus / MQTT edge network telemetry bridging.